

Project Report

Colorectal Cancer Survival Prediction

Data Mining II
Spring Semester 2024/2025

Group 33

Bruno Chaves - 20240370
Patrick Perlinger - 20240378
Rodrigo Santos - 20240389
Emmanuel Tronche-Macaire - 20240385

Abstract

This project aims to predict colorectal cancer patient survival using supervised machine learning, addressing a major global health issue. Applying the CRISP-DM methodology, we worked with over 75,000 patient records containing categorical and numerical medical data. Our objective was to build models that distinguish between survivors and non-survivors.

We performed extensive preprocessing, including handling missing data, resolving inconsistencies, normalizing features, and engineering new ones such as comorbidity count, tumor size categories, and age groups.

Despite these efforts, most initial models—from Logistic Regression to ensemble methods—predicted only one class, leading to deceptively high F1 scores but poor actual discrimination. We addressed this with hyperparameter tuning and more complex models like multi-layer perceptrons and stacking, but saw only marginal gains. Correlation analyses and Recursive Feature Elimination revealed weak, inconsistent predictors, pointing to limitations in data quality rather than model design.

Final model selection, based on Kaggle test performance, led us to deploy a stacking classifier. Our results underscore a key insight: high-quality, informative data is essential. Sophisticated models alone cannot overcome deficiencies in dataset structure or feature relevance.

Table of Contents

1. Introduction	3
1.1. Business Understanding	3
1.2. Previous Studies	3
2. Data Exploration and Preprocessing	4
2.1. Data Exploration	4
2.2. Data Preprocessing	5
3. Binary Classification	9
3.1. Baseline Model – OneRule Algorithm	9
3.2. Logistic Regression	9
3.3. Naive Bayes	9
3.4. Instance-Based Classifiers	10
3.5. Classification Tree	10
3.6. MLP Neural Network Classifier	10
3.7. Ensemble Models	11
3.8. Model Comparison & Evaluation	13
4. Open-Ended Section	14
4.1. Interpretation of Predictions	14
4.2. Interpretation of Feature Importance	14
4.3. Interpretation of the Dataset	15
5. Conclusion	15

1. Introduction

1.1. Business Understanding

The problem we are aiming to solve with our project is revolving around the mortality caused by colorectal cancer. Every year millions of people worldwide are affected by this disease and our task is to predict the cancer survival of new patients based on the data that was made available to us. Therefore, we want to facilitate early detection and the discovery of key risk factors. Our mission in this project is nothing less than saving the lives of thousands or even millions of people around the globe.

1.2. Previous Studies

The documents published on colorectal cancer survival prediction are close to a thousand per year.

Two important studies are listed below:

1. Yang, Xulin, et al. (2023) - "Predicting Colorectal Cancer Survival Using Time-to-Event Machine Learning: Retrospective Cohort Study"¹: the study stands out by using *time-to-event* machine learning models (like Random Survival Forest and Gradient Boosting Machine). Many earlier ML studies simplify survival prediction into a binary classification (Yes/No), losing crucial information about the timing of events. Time-to-event models are more robust and clinically relevant.
2. Mzoughi, Hela, et al. (2021) - "Predicting Colorectal Cancer Recurrence and Patient Survival Using Supervised Machine Learning Approach"²: this study predicts both recurrence and survival. Recurrence is a critical intermediate endpoint that heavily influences long-term survival. Predicting both gives a more holistic understanding of the patient's disease trajectory. The study is based in South Africa, introducing different patient populations, genetic backgrounds, and healthcare contexts, compared to traditional Western-centric studies.

The five most relevant variables, based on the two studies mentioned above, are the following:

- **Cancer Stage** (Metastatic, Regional, Localized): This is the most crucial prognostic factor. Localized cancer has higher survival rates than the other two
- **Early Detection**: this variable is associated with "Cancer Stage", since earlier detected cancer tends to be Localized. For this reason, it constitutes an important predictor.
- **Genetic Mutation**: it refers to changes or alterations in the DNA sequence of genes within the cancer cells. This is a crucial factor because specific mutations can drastically alter the disease's aggressiveness and, crucially, a patient's response to targeted therapies or immunotherapies.
- **Treatment Type** (Surgery, Chemotherapy, Radiotherapy, Combination): The chosen treatment strategy is directly aimed at affecting the disease course and, therefore, survival.
- **Tumor size (mm)**: Larger tumors are generally associated with more advanced disease, a higher likelihood of local invasion, and increased risk of spread, all of which negatively impact survival.

¹ <https://www.jmir.org/2023/1/e44417/>

² <https://www.frontiersin.org/journals/public-health/articles/10.3389/fpubh.2021.694306/pdf>

2. Data Exploration and Preprocessing

2.1. Data Exploration

2.1.1. General Insights

In our project we are dealing with a dataset of 75.035 rows and 32 columns. Out of these 32 columns 26 of them are strings (all categorical data), 5 are decimal numbers (whereas 1 of them - "Marital Status" - doesn't have any values) and one is an integer (the ID).

It contains data of patients from different countries and of different ages. While there is some demographic information provided, most of the variables display different aspects of the patients' medical information.

2.1.2. Missing Values & Duplicated Rows

We realized that the data quality is not exactly perfect, as we are missing values in almost every column. Besides the column "Marital Status", which is completely empty, the missing values are less than 100 for each of the columns, which is not a significant amount considering that our dataset has 75.035 rows. This is very positive, as it means we will have almost no imputed data, and our model will be trained almost exclusively on real data. When checking for duplicated rows, we realized that there are 15 duplicates in our dataset, which means we will have to consider this aspect during data preprocessing.

2.1.3. Distributions & Outliers

2.1.3.1. Numeric Variables

When looking at the distributions of our 4 numeric variables, we could see that all of them follow a similar pattern. They follow a rather dense distribution around the median (two of them looking more like a normal distribution, the other two more like a uniform distribution) and they all have only a few outliers that are all very distant from the whiskers of the boxplot. Interestingly the variables "Healthcare Costs" and "Mortality Rate per 100K" contain outliers that have negative values. These will have to be treated appropriately during the data preprocessing.

2.1.3.2. Categorical Variables

To get an understanding for the distribution of the categorical variables we used countplots. Some of the most relevant insights gained from these countplots are:

- We are dealing with a rather balanced dataset, as the distribution of the target variable is around 60/40.
- Some variables need to be preprocessed before they can be used (e.g. Reduce the "Genders" from 3 to 2; Join the "?" in "Healthcare Access" to another category; Join the capitalized and non-capitalized categories in "Urban or Rural").
- Some variables will for sure not be useful for our model ("Marital Status" due to missing values, "Transfusion History" due to only one unique value, "Diabetes History" due to extreme imbalance of the two unique values).

2.1.4. Correlations & Associations

2.1.4.1. Numeric to Numeric Variable

In order to detect correlations between the four numeric variables, we used the Pearson and the Spearman Correlation Matrix. Both methods came to the same result that there are clearly no correlations between any of these variables.

2.1.4.2. Categorical to Categorical Variable

Using the Chi-Squared Test, we found that within all the 276 possible bivariate combinations, there are 9 correlations that we consider significant due to a p-value smaller than 0.05. Out of these 9 correlations, one correlation is to the target variable ("Treatment Type" - "Survival Prediction"). This gives us the reason to assume that "Treatment Type" might be a very relevant variable for our final model.

2.1.4.3. Numeric to Categorical Variable

To understand the correlations between the numeric and the categorical variables, we used the One-Way ANOVA Test. This test resulted in 7 significant correlations (p-value < 0.05), which are all about the influence of a healthy lifestyle (not smoking, no alcohol consumption) or of early cancer detection on mortality, incident rate and healthcare costs. Even though the correlations are statistically significant, we can see that the actual percentage of the difference between the means of the different categories is very small in each of the 7 correlations. Therefore, we can conclude that the low p-value is mainly caused by the big sample sizes and not so much by huge differences in the values in each category.

2.2. Data Preprocessing

2.2.1. Dataset split

The method we will use to split the dataset is the hold-out method, dividing the dataset of labeled data into three separate sets: 70% of the rows will form the training set X_{train} (52.523 observations), 15% the validation set X_{val} (11.256 observations) and 15% the test set X_{test} (11.256 observations). According to our evaluation and data understanding, the size and type of data does not require k-fold cross-validation.

We also have three corresponding subsets with the target variable only as column, and the rows split follows the exact same pattern as the three sets of the independent variables, ensuring correspondence between the rows of the two sets. These subsets are called y_{train} , y_{val} and y_{test} .

2.2.2. Preliminary inconsistencies

2.2.2.1 Initial Analysis

A preliminary review of the categories classified for each categorical variables leads us to review some of the aggregations and category names:

- **Gender:** the variable includes three categories M, F, and P. While 'P' makes sense in demographical terms, in statistical terms the number of observations is insufficient, and we re-assign the value 'P' to the mode 'M', assuming over 50% of the observations will be imputed correctly based on gender assigned at birth.
- **Healthcare Access:** some rows have value '?', thus we impute them as missing values
- **Smoker:** we have two duplicate columns including smoking habits, 'Non Smoker' and 'Smoking History'. While the latter makes more common sense because "Yes" means smoker and "No" means non-smoker, the former is the only one which has no missing values, so using this one provides more

accurate information. However, as it is now, smokers are labeled as "No" and non-smokers as "Yes", which can be confusing. To make it easier to read, we will rename the variable "Smoker" and invert the responses.

- **Transfusion History:** all rows have value '-', thus we impute all as missing values
- **Urban or Rural:** some categories are in capital letters, others not. We will re-assign 'urban' to 'Urban' and 'rural' to 'Rural' to ensure we only have two categories.

2.2.2.2 Duplicated rows

We found 15 duplicated values affecting 30 rows in total. These duplicates are equal in every column except of the column "ID". Therefore we decided to remove them.

2.2.3. Missing values

Most of the variables show missing values. We will use four different methods to handle them:

1. **Column removal:** two variables have complete lack of data and are removed. (*'Marital Status', 'Transfusion History'*);
2. **Imputing the median:** all four continuous numerical variables have missing data for < 0.1% of the datasets. Therefore, we can safely impute the median without altering too much the distribution. (*'Healthcare Costs', 'Incidence Rate per 100K', 'Mortality rate per 100K', 'Tumor Size (mm)'*);
3. **Imputing the mode:** for categorical variables that have a clear mode, imputing the mode is likely to be more accurate than imputing at random. We define a clear mode as a distribution where the majority class has over 40% more observations than any of the other categories. (*'Country', 'Diabetes', 'Diet Risk', 'Early Detection', 'Family History', 'Gender', 'Genetic Mutation', 'Healthcare Access', 'Inflammatory Bowel Disease', 'Insurance Status', 'Treatment Type', 'Urban or Rural'*);
4. **Imputing at random:** for categorical variables without a clear mode – i.e. where the majority class does not have at least 40% more observations than any other category - even though missing values account for less than 0.1% of the rows in the datasets, imputing to the mode could be distorting especially for those variables with many different categories, such as 'Country' and 'Date of Birth'. For this reason, the values will be imputed at random. (*'Alcohol Consumption', 'Cancer Stage', 'Date of Birth', 'Obesity BMI', 'Physical Activity', 'Screening History'*).

2.2.4. Outlier removal

With regards to continuous numerical variables, the univariate analysis through boxplots reveals a low number of outliers and in all four cases we can identify a clear-cut point using the whiskers of the boxplot ($1.5 * IQR$). For two variables we only consider the upper whisker (*'Incidence Rate per 100K', 'Tumor Size (mm)'*) and for the other two we consider both whiskers (*'Healthcare Costs', 'Mortality Rate per 100K'*).

While negative values clearly suggest a wrong data entry, for other values we might think of other hypothesis such as the use of a different unit of measure or an extra zero at the end. But given the low number of rows affected (< 0,05% of total rows), we can safely remove those without significantly impacting the quality of data.

2.2.5. Encoding

The purpose of this crucial step is to transform the text data of our categorical variables into a datatype that can be later processed correctly during the model selection and training. We will follow five different approaches depending on the nature of the variables.

2.2.5.1. Column removal

In '*Diabetes History*' the data is too imbalanced (52460 'No' vs 8 'Yes'). We do not want to encode the variable with such imbalance. Additionally, reassigning 'Yes' to the mode does not make sense either in a binary variable, as there would only be one category. For this reason, the column is dropped.

2.2.5.2. Binary encoding

There are ten binary variables with Yes/No categories, including the target variable '*Survival Prediction*'. For all of these, we encode 'No' as 0 and 'Yes' as 1.

2.2.5.3. One-hot encoding

For categorical variables with more than two categories ('*Treatment Type*') and binary variables that are not of Yes/No type ('*Gender*', '*Insurance Status*', '*Urban or Rural*'), where 0 and 1 can be less intuitive to impute, we apply one-hot encoding. It will be generating new columns - precisely one less than the number of different categories that the variable shows - and we apply 1 when the category mentioned in the column header is the one reflected in the observation. When all columns representing that feature are 0 it means the category is the only one that does not have a column of its own, by a process of elimination.

2.2.5.4. Ordinal encoding

There are other non-binary variables that we have not considered yet. These have more than two categories but can be reasonably ordered based on common sense, so we do not need one-hot encoding with the addition of new columns, but we can order the values in increasing order from 0 onwards instead. ('*Cancer Stage*', '*Diet Risk*', '*Healthcare Access*', '*Insurance Costs*', '*Obesity BMI*', '*Physical Activity*', '*Screening History*'). As an example, with '*Obesity BMI*' we convert 'Normal' = 0, 'Overweight' = 1 and 'Obese' = 2 in increasing order.

2.2.5.5. Average target encoding

For variable 'Country', which shows 16 different categories, we opt for average target encoding to avoid creating 15 additional columns with one-hot encoding. This means that for each row of that column, we will replace the country name by a number representing the average value of the target variable for that category.

2.2.6. Feature Engineering

In this step, we focused on engineering new features to improve our models predictive power and functionality. These are the new features engineered:

1. **"Age"**: Calculated as the difference between today's date and the column "*Date of Birth*".
2. **"Age_groups"**: Extracted from "*Age*" by performing binning, we defined the following thresholds for the groups: 0 for ages 0-30, 1 for ages 31-60, and 2 for ages 61-100.
3. **"Tumor Size Categories"**, extracted from "*Tumor Size (mm)*" also with binning: 0 for tumor sizes 0-35mm, 1 for sizes 36-45mm, and 2 for sizes 46-100mm. We followed the guidelines from researchers of the US National Center for Biotechnology Information.
4. **"Smoker and Alcohol Consumption"**: Boolean showing if a patient both smokes and consumes alcohol.
5. **"Comorbidity Count"**: A numerical feature representing the total number of existing conditions from the "*Diabetes*", "*Heart Disease History*", "*Hypertension*", and "*Inflammatory Bowel Disease*" columns. The possible values range from 0 to 4.

6. **"Mortality_to_Incidence_Ratio_per_100k"**: This is a numerical feature representing the ratio between Mortality Rate and the Incident Rate.
7. **"Family History and Genetic Mutation Interaction"**: Boolean if both *"Family History"* and *"Genetic Mutation"* are present.

We chose to create these new features, particularly the binned ones, with the specificities of our models in mind. For certain models, using binned variables can lead to better performance compared to the original continuous or metric versions (for example Tree-Based Models or Naive Bayes Classifiers). Ultimately, these new features will be tested during model training and evaluation, either by replacing existing variables or by serving as additive features.

2.2.7. Correlations & Associations - Preprocessing

After performing the feature engineering, we had another look into correlations and associations between different variables to understand if after the preprocessing of the data there are any significant changes visible. Besides some correlations between the engineered features and the features they were derived from, there were no new insights gained compared to the Data Understanding chapter.

2.2.8. Data Normalization

As one of the final steps for data pre-processing, we perform data standardization using Z-score (Z-transformation), this method is preferred because it handles better the outliers and performs better with some of the linear models. This method consists of subtracting the mean to each observed value and dividing it by standard deviation, with the final result being the Z-score.

The calculations were performed solely to the following metric columns: *"Healthcare Costs"*, *"Incidence Rate per 100K"*, *"Mortality Rate per 100K"*, *"Tumor Size (mm)"*, *"Age"* and *"Mortality to Incidence Ratio per 100k"*.

2.2.9. Feature Selection

On this step, we defined the variables that can be used in the modelling step. After interpretation of correlations and association, we did not find any significative evidence of columns that should be removed, only columns that cannot be predictors together, so we got to the conclusion that we need more tools to improve our feature selection. To achieve that objective, we performed RFE analysis optimized for Logistic Regression and Random Forest, purely as indicators of what should be the features selected, and we got the following results:

The 10 best features for Logistic Regression are: *"Family History"*, *"Insurance Status_Uninsured"*, *"Urban or Rural_Urban"*, *"Treatment Type_Combination"*, *"Treatment Type_Radiotherapy"*, *"Treatment Type_Surgery"*, *"Country enc"*, *"Age_groups"*, *"Tumor Size Categories"*, *"Family History and Genetic Mutation Interaction"*.

While the 10 best features for Random Forest are: *"Healthcare Costs"*, *"Incidence Rate per 100K"*, *"Insurance Costs"*, *"Mortality Rate per 100K"*, *"Obesity BMI"*, *"Tumor Size (mm)"*, *"Country enc"*, *"Age"*, *"Comorbidity Count"*, *"Mortality_to_Incidence_Ratio_per_100k"*.

Before this step there were already feature removal and selection due to other reasons previously mentioned. Recognizing that each model should have its own optimal set of features (the best predictors for that specific model), we will conduct further feature selection as needed.

3. Binary Classification

3.1. Baseline Model – OneRule Algorithm

We chose to use a simple OneRule Algorithm as our Baseline Model, which will serve as the benchmark for more sophisticated algorithms that follow.

When examining the performance metrics and the confusion matrix, we can see a somehow strange and unexpected pattern here. The Recall value of 1.0 seems very unnatural and the confusion matrix indicates that there is something wrong, as the model always predicts class 1 on the validation set, even if the dataset is not very unbalanced. Also, the fact that the results between train and validation set are practically identical brings up some doubts about the performance of this model.

Performance Metrics:						Confusion Matrix:		
Data	Accuracy	Precision	Recall	F1 Score	ROC AUC		Predicted 0	Predicted 1
Train	0.6005	0.6005	1.0	0.7504	0.5	Actual 0	0	4494
Val	0.6006	0.6006	1.0	0.7505	0.5	Actual 1	0	6759

3.2. Logistic Regression

After our Baseline Model, we used another simple algorithm to see how the results and the performance change compared to the OneRule Algorithm. For this, we chose to use Logistic Regression.

Against our expectations, the Logistic Regression performs the same way as the OneRule Algorithm. The confusion matrix shows the same problem of predicting everything as class 1 and the performance metrics are almost identical, only the ROC AUC curve shows some minor improvement compared to our Baseline Model.

Performance Metrics:						Confusion Matrix:		
Data	Accuracy	Precision	Recall	F1 Score	ROC AUC		Predicted 0	Predicted 1
Train	0.6005	0.6005	1.0	0.7504	0.5095	Actual 0	0	4494
Val	0.6006	0.6006	1.0	0.7505	0.5105	Actual 1	0	6759

3.3. Naive Bayes

After receiving somewhat dissatisfying results from our Baseline Model and the Logistic Regression, we chose to use probabilistic classifiers and train a Naive Bayes Algorithm on our data.

Even after the third model, the same pattern continues. Again, the model predicts every instance as class 1, and all metrics are the same as before besides the ROC AUC curve (which again slightly improved compared to the Logistic Regression).

After training 3 models, we started to get the notion that there might be something wrong with the preprocessing of our data or about the dataset itself.

Performance Metrics:						Confusion Matrix:		
Data	Accuracy	Precision	Recall	F1 Score	ROC AUC		Predicted 0	Predicted 1
Train	0.6005	0.6005	1.0	0.7504	0.5131	Actual 0	0	4494
Val	0.6006	0.6006	1.0	0.7505	0.5212	Actual 1	0	6759

3.4. Instance-Based Classifiers

After achieving, once again, unsatisfactory results Naive Bayes model, we tested an Instance-Based Algorithm: the Nearest Neighbour Classifier, specifically the Ball Tree. Unlike previous models, confusion matrix showed different predictions, but these can't be called improvements. In fact, this model is not so biased towards the class 1, however the model performance didn't show improvements.

The F1-score decreased compared to previous models (even when comparing the optimal "k" of neighbours), and the ROC AUC underperformed comparing to a Random Model.

Performance Metrics:						Confusion Matrix:		
Data	Accuracy	Precision	Recall	F1 Score	ROC AUC		Predicted 0	Predicted 1
Train	0.6069	0.6209	0.8869	0.7304	0.5910	Actual 0	632	3862
Val	0.5738	0.6012	0.8628	0.7086	0.4972	Actual 1	946	5813

3.5. Classification Tree

As seen for other models before, the classification tree also tends to project almost all – if not all – predictions on the "Survival" category. Pruning helps make the model less computationally expensive and is crucial to reduce overfitting, but none of the modifications actually improve the model performance. For this reason, the best solution is to keep the model as simple as possible:

- only one variable, "Healthcare Costs",
- only one decision, a numerical threshold on "Healthcare Costs",
- a value of maximum depth between 1 and 3. In the final version we used 1 to have the simplest possible model.

Results are extremely similar between train and validation set and can be summarized as follows:

Performance Metrics:						Confusion Matrix:		
Data	Accuracy	Precision	Recall	F1 Score	ROC AUC		Predicted 0	Predicted 1
Train	0.6005	0.6005	1.0	0.7504	0.5000	Actual 0	27	4467
Val	0.6016	0.6015	0.9976	0.7505	0.5018	Actual 1	16	6743

3.6. MLP Neural Network Classifier

As our final implementation when it comes to single models, we implemented a multi-layer perceptron classifier, being a more complex approach, we expect that now we can find a model that understands the extremely complex patterns present in our dataset. These are the key notes that we could take from our implementation:

- Best Parameters are: 'solver':'adam', 'max_iter':1200, 'learning_rate':'constant', 'hidden_layer_sizes': (100,), 'alpha': 0.001, 'activation': 'relu'.

- The average balanced accuracy across cross-validation folds was 0.503 (very low, considering that 0.5 is random guess for binary classification problems).

Results from MLP Classifier:

Performance Metrics:						Confusion Matrix:		
Data	Accuracy	Precision	Recall	F1 Score	Balanced Accuracy		Predicted 0	Predicted 1
Train	0.64	0.66	0.83	0.7355	0.592	Actual 0	1132	3362
Val	0.56	0.61	0.76	0.6753	0.503	Actual 1	1600	5159

3.7. Ensemble Models

With our previous models as well as with ensemble models without tuning, we continued to verify existing problem with our predictions, with most of the models only predicting one class. To address this problem, we decided to optimize our models by defining a hyperparameter tuning function ("*hyperparameter_tune*") with a purpose of finding the best hyperparameters for our machine learning model using random search with cross-validation. Key notes about our function:

- We are using stratified K-folds to preserve the class distribution.
- We also use *RandomizedSearchCV* to sample random hyperparameter settings from parameters, optimizing for balanced accuracy (noting that we also did other experiments here, e.g. Weighted F1-score or accuracy).
- The output here is the elapsed time for each search, mean and standard deviation of cross-validation scores and finally the best score and corresponding parameters.

The choice for randomized search CV was because we noticed a lot of noise in our dataset. This function is used for *Random Forest*, *Extra Trees*, *Adaboost*, *Gradient Boosting* and *MLP Classifier*.

3.7.1. Bagging: Random Forest Classifier & Extra Trees Classifier

Without tuning, *Random Forest* and *Extra Trees*: Both models are heavily biased toward class 1, rarely identifying any class 0 correctly. Possible reasons include class imbalance or uninformative features. With tuning, we should address this issue by working to reduce uninformative features and, where possible, balance the data.

After performing the hyperparameter tuning, we found these as the best hyperparameters:

1. Random Forest: 'n_estimators': 400, 'min_samples_leaf': 24, 'max_features': 'sqrt', 'max_depth': 10, 'criterion': 'entropy', 'class_weight': 'balanced_subsample', 'bootstrap': True.
2. Extra Trees: 'n_estimators': 100, 'min_samples_leaf': 42, 'max_depth': 5, 'criterion': 'gini', 'class_weight': 'balanced_subsample'.

Results from Random Forest:

Performance Metrics:						Confusion Matrix:		
Data	Accuracy	Precision	Recall	F1 Score	Balanced Accuracy		Predicted 0	Predicted 1
Train	0.64	0.73	0.65	0.6887	0.6421	Actual 0	1736	2758
Val	0.53	0.61	0.63	0.6177	0.511	Actual 1	2506	4253

Results from Extra Trees:

Performance Metrics:						Confusion Matrix:		
Data	Accuracy	Precision	Recall	F1 Score	Balanced Accuracy		Predicted 0	Predicted 1
Train	0.55	0.65	0.56	0.6011	0.5496	Actual 0	1994	2500
Val	0.53	0.61	0.59	0.6011	0.502	Actual 1	2780	3979

Looking at the results, we understand that using '*class_weight*': '*balanced_subsample*' or '*balanced*' made the Random Forest and Extra Trees predict some 'class 0' correctly. However, we kept very high error rates.

3.7.2. Boosting: Ada Boost Classifier & Gradient Boosting Classifier

Without tuning with *AdaBoostClassifier* and *GradientBoostingClassifier*, we again observe that both models predict nearly all samples as class 1, effectively ignoring class 0. Additionally, our experiments show that if we try to optimize these models for class 0 recall, they end up predicting everything as class 0. Similarly to the Bagging Models, we tried to mitigate these problems with hyperparameter tuning, but in the case of the Boosting Models even after tuning they heavily leaned towards predicting class 1.

After performing the hyperparameter tuning, we found these as the best hyperparameters:

1. *AdaBoostClassifier*: '*n_estimators*': 150, '*learning_rate*': 0.009.
2. *GradientBoostingClassifier*: '*subsample*': 1.0, '*n_estimators*': 150, '*min_samples_split*': 10, '*min_samples_leaf*': 2, '*max_depth*': 8, '*learning_rate*': 0.02.

Results from AdaBoost:

Performance Metrics:						Confusion Matrix:		
Data	Accuracy	Precision	Recall	F1 Score	Balanced Accuracy		Predicted 0	Predicted 1
Train	0.60	0.60	1.00	0.7504	0.5	Actual 0	0	4494
Val	0.60	0.60	1.00	0.7505	0.5	Actual 1	0	6759

Results from Gradient Boosting:

Performance Metrics:						Confusion Matrix:		
Data	Accuracy	Precision	Recall	F1 Score	Balanced Accuracy		Predicted 0	Predicted 1
Train	0.6201	0.61	0.99	0.76	0.525	Actual 0	52	4442
Val	0.5986	0.60	0.99	0.75	0.501	Actual 1	75	6684

3.7.3. Stacking

Here, we are implementing a model stacking solution setting up the models with the best parameters from previous model tuning, trying to use a more complex methodology in a complex problem. We divided this process in multiple steps:

1. Training Individual (Base) Classifiers: Random Forest, Extra Trees and MLP Classifier.
2. Create a Stacked Classifier (final predictions are made by a Random Forest that learns from the outputs of the three base models).
3. Making Predictions & Evaluate the model.
4. Evaluate the Metrics.

Results from Stacked Classifier:

Performance Metrics:						Confusion Matrix:		
Data	Accuracy	Precision	Recall	F1 Score	Balanced Accuracy		Predicted 0	Predicted 1
Train	0.6554	0.74	0.66	0.70	0.6547	Actual 0	2111	2383
Val	0.5138	0.61	0.54	0.57	0.507	Actual 1	3088	3671

3.8. Model Comparison & Evaluation

3.8.1. Model Comparison

When we look at the metrics of all our models, we can see very surprising results that we would not have imagined at the beginning of the project. The simplest models perform the best on the most relevant metric in this project - the F1 score. Even our simple OneRule Algorithm reaches a higher F1 score than a MLP Neural Network with optimized hyperparameters.

This strange pattern occurs due to problems in the dataset as we understood during the modelling process. Because of that, the best performance is reached when every instance in the validation set is predicted as class 1 "Survival".

While under normal circumstances we would choose the best 2-3 models according to F1 score at this stage, train them on the full dataset and then make predictions on the test set, these circumstances call for different solutions, as we have seen in our first Kaggle submission with a model that classifies everything as class 1 that the model performance is weaker than the performance of our peers.

Therefore, we selected not only the best models according to the F1 score to train on the full dataset and make predictions, but also included models with weaker performance which do not only classify every instance as class 1.

Eventually we submitted the predictions of all of these selected models on Kaggle and chose the one that scores the best on the test set as our final predictor.

3.8.2 Kaggle submissions

The models of our choice to try for the Kaggle submission are:

1. Naive Bayes:

As it's one of the models that has the highest F1 score (0.7505; predicting everything as class 1).

2. MLP Neural Network:

As among the models that classify a significant amount of instances as class 0 it shows one of the best F1 score performances (0.6753) and generally Neural Networks are well suited to solve complex problems.

3. Stacking Model:

Even if it doesn't show a very convincing performance on the validation set (F1 score: 0.57), we want to try this model on Kaggle as it predicts class 0 more frequently than other models and its complexity might help to solve the problem we are facing better than other models.

4. Hyperparameter-tuned Random Forest:

As among the hyperparameter-tuned ensemble classifiers it performs the best (given that class 0 is predicted a significant number of times; F1 score: 0.6177).

3.8.3 Final Model

After training these 4 models on the entire dataset, obtaining their predictions on the test set and uploading these predictions on Kaggle, we received the following scores for each model:

1. **Naive Bayes:** 0.45360
2. **MLP Neural Network:** 0.52743
3. **Stacking Model:** 0.52899
4. **Hyperparameter-tuned Random Forest:** 0.52154

Due to the scores obtained from Kaggle, we chose our Stacking Model to be our final model that we are going to deploy to predict survival chances for colorectal cancer patients.

4. Open-Ended Section

4.1. Interpretation of Predictions

Despite testing both simpler and more complex models, our main finding is that increasing the models' complexity didn't improve the models' performance on regards to the F1 score. In fact, MLP and other more complex models optimized with hyperparameters performed worse than our OneRule Algorithm in this key metric.

All models we implemented have consistently shown patterns of classification imbalance. From the OneRule Algorithm to more complex models, the output was a strong bias towards predicting 1 (Survival). This resulted in having a high recall (1 or close to 1) and low precision, resulting in a satisfactory F1 score, which could mislead our analysis if weren't by the previous findings.

On section 3.8, after comparing the models, we conclude that our roof for this metric (approximately 0.75), is interestingly achieved when all predictions were 1. Contradicting what is expected, since the more complex and balanced predicting models lose too much recall when increasing their precision. Even stacking (see 3.7.3), that combines the strengths of each model, did not lead us to a better performance, remaining with same performance issues we had before.

Since all predictions in the initial models were 1, and this class represent approximately 60% of the dataset, automatically the Accuracy (% of correctly predicted cases) was also 60%, not being a good indicator of the predictability. Also, the ROC AUC has contributed to the same conclusions – all models had a hard time to outperform a random model, since our results only slightly perform above 0.5.

4.2. Interpretation of Feature Importance

As we concluded in section 2.1.4, there are not only weak correlations between the features, but also a lack of relevancy. Both Pearson and Spearman tests indicated weak relationship between numerical features, while the Chi-Squared Test showed some improvement with the categorical variables, presenting a few significant results. Lastly, ANOVA also indicated some significant correlations, however, these can likely be disregarded due to the large sample size.

On the section 2.2.9, we also concluded from the RFE analysis interesting but contradictory results, with different top features selected by Logistic Regression and Random Forest. This raised our concern about the

reliability of the data, as we would expect that strong predictors would be consistent independently of the chosen model.

These discrepancies only reinforced our lack of confidence. Combining weak correlations with inconsistent features selection suggests that our data lacks relevant features, limiting its predictive power from reaching its full potential. Additionally, these issues also show that there may be underlying concerns with the data itself, that are contributing to the absence of strong predictors in our models.

4.3. Interpretation of the Dataset

The consistently underperforming models would suggest that there are underlying issues on the dataset or preprocessing steps. As we believed that we made a carefully execution of the preprocessing steps, combining with more complex modelling approaches, our conclusion is that the issues consist of the dataset itself.

Additional statistic evidence supports our conclusions. When we analysed tumor size distribution among survival prediction, we faced an even distribution between the two classes. It is expected that the survival probability reduces as cancer tumor increases (which is also emphasized by previous studies mentioned in chapter 1.2), however our correlation tests did not find any correlation between this feature and the target. It is also well known that unhealthy habits as smoking and alcohol consumption or diseases like diabetes and obesity can negatively contribute to the survival prediction. And none of those showed any relevancy in relation to the target variable.

The variety of models trained with low performance, added with the inconsistencies in the data and features all points to the same direction: the dataset quality is low. As we know good models are only possible when fed with good, informative and uncontradictory data. In our case, our dataset seems not to suit in these criteria. This reinforces the notion that data quality is a critical factor for achieving better predictive performance.

5. Conclusion

The primary goal of our project was to predict colorectal cancer survival using supervised learning techniques, following the CRISP-DM framework. We aimed to identify key risk factors and improve early detection by building effective predictive models on a dataset of over 75,000 patients. After extensive data preprocessing, feature engineering, and model training—including baseline models, ensemble methods, and a neural network—we found that most models exhibited a strong bias toward predicting a single class. While this resulted in deceptively high recall and F1 scores, it revealed a critical issue: the models struggled to learn meaningful patterns from the data.

Contrary to our expectations, model complexity did not translate into improved performance on the validation set. Even with hyperparameter tuning and advanced techniques like stacking, results remained limited. This highlighted a major limitation: the dataset itself, which lacked strong correlations between features and the target variable and possibly suffered from poor data quality or insufficient feature diversity. Future work should focus on improving data quality, exploring resampling techniques (e.g., SMOTE), or acquiring more informative features. Alternative approaches like time-to-event modelling could also provide more nuanced insights into survival outcomes.